Lecture 05 - Interfaces and Generics

Functions

```
func add(a, b int) (int, error) {
    return a + b, nil
}

func main() {
    num1 := 5
    num2 := 7

    result, _ := add(num1, num2)
}
```

Closure

```
func main() {
    count := 0

    increment := func() {
        count++
    }

    increment()
    increment()
    increment()

    fmt.Println(count) // 3
}
```

```
func multiplier(factor int) func(int) int {
    return func(value int) int {
        return value * factor2
    }
}

func main() {
    double := multiplier(2)
    triple := multiplier(3)
    tenTimes := multiplier(10)

    fmt.Println(double(5)) // 10
    fmt.Println(triple(7)) // 21
    fmt.Println(tenTimes(7)) // 70
}
```

```
try:
    ...
except:
    ...
```

```
type error interface {
    Error() string
}
```

```
import errors


func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("cannot divide by zero")
    }

    return a / b, nil
}
```

```
func main() {
    x := 5
    y := 0

    result, err := divide(x, y)

    if err != nil {
        slog.
        log.
        fmt.Print("error happened:"
    }

}
```

```
var ErrUserNotFound := errors.New("user not found")
var ErrUserAlreadyExists := errors.New("user not found")
...
```

```
exceptions
    UserNotFoundException.php
    ...
```

/user/<:id>

```
func findUser(userID int) User {
    user, err := userRepository.find(userID)

    if errors.Is(err, ErrUserNotFound) {
        return fmt.Errorf("user not found: %w", err)
    }

    if errors.Is(err, errors.New("timeout problem")) {
        ...
        // log
        // metric
        // retries
    }

    return user, nil
}
```

```
func processFile() {
    file, err := os.Open("data.csv")
    defer file.Close()
    defer fmt.Println(1)
    defer fmt.Println(2)

    if err != nil {
        return err
    }

    // processed
    calculateSomething(file)

    file.Close()
}
```